You said:

also "code" in the endpoint interface is a little vague isn't it?

ChatGPT said:

Yes. `code` is developer/database language leaking into the customer interface.

A professional customer is more likely to recognize something like:

- **Endpoint ID**
- **Site ID**
- **Location ID**
- **Store / Branch ID**
- **Customer location ID**

Which label is best depends on how broad we want Depot to be, and I think **Endpoint ID** is probably the safest generic term.

Then the UI can explain it once:

> **Endpoint ID** — your existing identifier for this store, clinic, branch, warehouse, office, or other supported location.

Internally we can still store the field as `code` if we want, but the customer should not have to infer what "code" means.

The import mapping should also use customer-facing terminology:

```
Endpoint ID
Endpoint name
Address
Site type
```

```
Contact
Access notes
```

rather than:

```
code
name
address
```

And for your existing CSV, `STORE NUMBER` is clearly the natural match for **Endpoint ID**.

This is another example of the same broader principle we're uncovering in the walkthrough:

Depot should adapt to the customer's operational vocabulary, not expose our internal schema vocabulary.

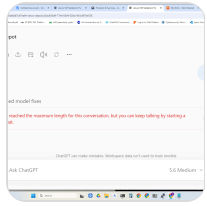
I would change `code` to **Endpoint ID** everywhere in the client-facing UI and templates.

You said:

Ok so let's fix it

Thinking

You said:



Worked for 22s

ChatGPT said:

It looks like the fix has already been pushed to the rebuild branch.

The current branch head is:

`a1e060f49bf84f79a2f0f6420162b10683dec546`
"Clarify endpoint identity and recovery policy ownership"

And I verified the actual code now reflects the changes we just discussed:

- client-facing `code` is now **Endpoint ID**
- endpoint import no longer requires an RTO/recovery target
- realistic location fields are supported: site type, address lines, city, state, ZIP, country, market, contact, phone/email, access notes, enabled/active
- the importer now has aliases such as `STORE NUMBER → Endpoint ID`, `STORE NAME → Endpoint name`, `STREET → address line 1`, etc.
- recovery target moved to **supported equipment** as optional `recoveryTargetHours`
- tests were added specifically for realistic endpoint CSV imports and for recovery targets belonging to equipment rather than endpoints

So on the VM, just pull the rebuild branch:

```
cd /opt/cesco/depot-rebuild
git pull --rebase


cd app
npm test
npm run test:http
npm run build
```

Then restart the port-3001 dev instance if needed:

```
npm run dev -- --port 3001
```

After refresh, upload the **same old endpoint CSV again**.

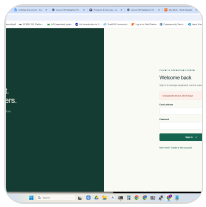
What I expect this time is that the mapping screen should automatically infer most of your existing columns, especially:

```
STORE NUMBER   → Endpoint ID
STORE NAME     → Endpoint name
LOCATION TYPE  → Site / location type
STREET         → Street / address line 1
SUITE          → Suite / address line 2
CITY           → City
STATE          → State / province
ZIP            → ZIP / postal code
COUNTRY        → Country
MARKET         → Market / region
CONTACT        → Site contact
EMAIL          → Contact email
PHONE          → Contact phone
NOTES          → Access / delivery notes
ENABLED        → Active / enabled
```

And there should be **no recovery-target field on the endpoint import at all**.

That is exactly the behavior we wanted.

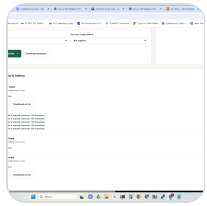
You said:



Can't even login

Thinking

You said:



I was able to upload the endpoint csv then got errors when I uploaded the asset csv

Inspect CSV Header Mapping Logic

Connection interrupted. Waiting for the complete answer